Experiment II — Advanced IaC Techniques

Submitted by-Shreyash Shivhare
Sap ID-500122740
Roll no.-R2142231555
Batch-2 (DevOps)

Field	Detail
Course outcome	CO5
Marks	5 marks (part of the 20 marks allotted to CO5)
Date of performance	_____
Marks awarded	_____

Aim

To explore advanced Infrastructure as Code capabilities by provisioning multiple interdependent virtual machines and by configuring network settings and security groups entirely through code.

Objectives

- To model a multi-tier network topology — VPC, subnets, gateway and routing — as code.
- To provision several virtual machines whose creation order is governed by dependencies.
- To express least-privilege firewall rules using security groups that reference one another.
- To refactor a flat configuration into reusable modules with defined inputs and outputs.
- To validate the design with both positive and negative connectivity tests.

Requirements

Hardware

Software

- Terraform CLI 1.6 or later
- AWS CLI v2, Git, Visual Studio Code
- Graphviz, for rendering the dependency graph produced by terraform graph
- An SSH client

Accounts and access

- The AWS account and IAM user from Experiment I, with permissions for EC2 and VPC
- An EC2 key pair created in the working region, for SSH access to the web tier

Theory

2.1 The Resource Dependency Graph

Terraform does not provision resources in the exact order they are written in the configuration file. Instead, it parses the configuration and builds a Directed Acyclic Graph (DAG) to determine the correct sequence of operations. If one resource references the attribute of another (an implicit dependency), Terraform guarantees that the prerequisite resource is created first. If a dependency cannot be inferred through references, the author must declare it manually using the depends_on meta-argument (an explicit dependency). This graph model allows Terraform to safely provision independent resources in parallel, drastically reducing deployment time.

2.2 Meta-Arguments: count and for_each

Infrastructure often requires multiple similar resources, such as a cluster of web servers. Rather than duplicating resource blocks, Terraform provides meta-arguments to handle iteration. The count meta-argument accepts a whole number and provisions that many identical copies of a resource, accessible via an index (e.g., count.index). The for_each meta-argument accepts a map or a set of strings, provisioning a distinct resource for each item. While count is simpler, for_each is preferred for infrastructure that may scale up or down dynamically, as it avoids the destructive shifting of array indexes.

2.1 Modules

A module is a container for multiple interrelated resources that are used together. Just as functions allow developers to reuse code in traditional software engineering, modules allow infrastructure engineers to package a complex topology—such as an entire VPC or a compute cluster—into a single, reusable component. Every Terraform configuration has at least one module, known as the root module. When the root module calls a child module, it passes arguments into the child's input variables and consumes the child's exported outputs.

2.2 Virtual Private Cloud Fundamentals

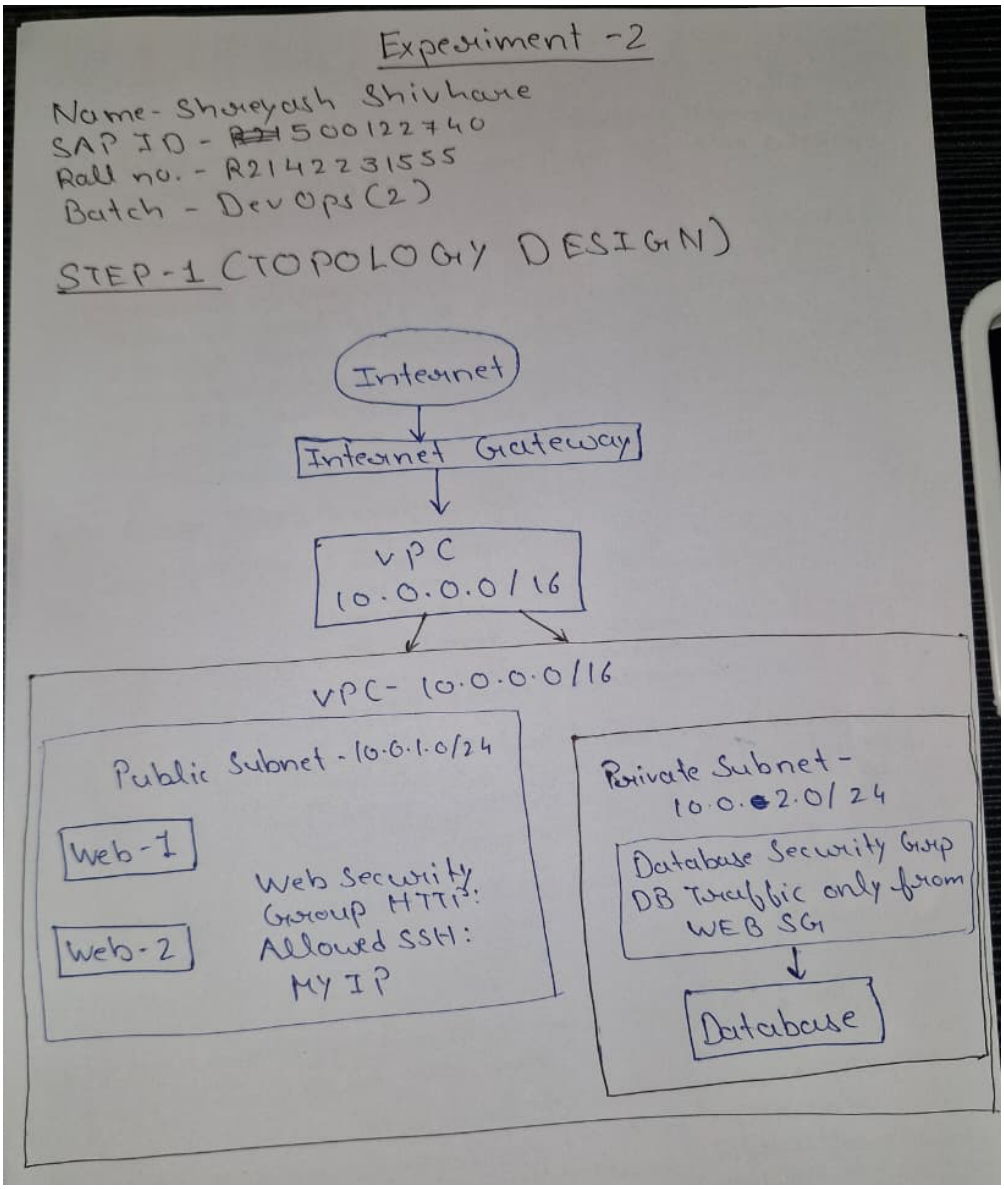
A Virtual Private Cloud (VPC) is a logically isolated section of the AWS cloud where cloud resources are launched in a custom-defined virtual network. A VPC spans an entire region and is carved into smaller chunks called subnets, which reside in specific Availability Zones. A public subnet is one whose traffic is routed to an Internet Gateway, allowing its instances to communicate with the outside world. A private subnet has no direct route to the internet, providing a secure boundary for backend resources like databases that must remain completely isolated from public access.

2.3 Security Groups and Least Privilege

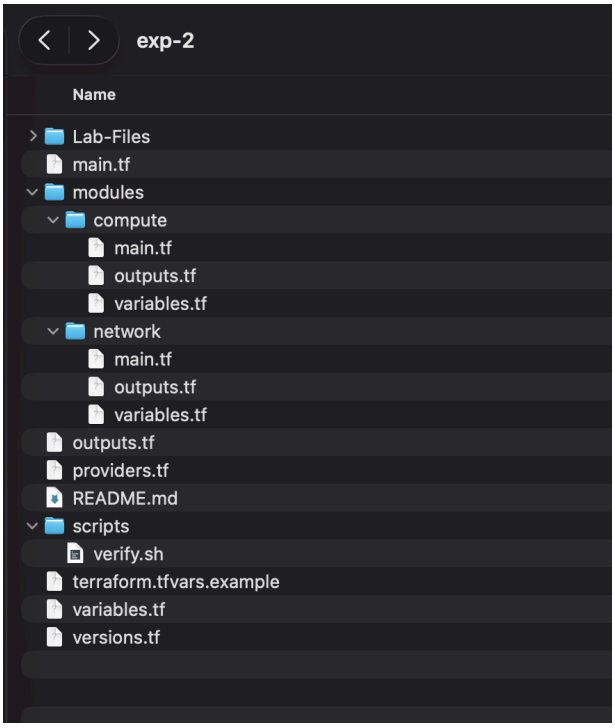
Security groups act as virtual, stateful firewalls operating at the network interface level to control inbound (ingress) and outbound (egress) traffic. They operate on a default-deny paradigm; all inbound traffic is blocked unless an explicit "allow" rule is authored. The principle of least privilege dictates that an entity should only be granted the exact permissions required to perform its function. In IaC, this is achieved by scoping security group rules as narrowly as possible—for instance, allowing database traffic on port 3306 exclusively from the security group ID of the web tier, rather than opening the port to a broad IP range.

Procedure

Step 1 — Design the topology on paper



Step 2 — Create the module directory structure



Step 3 — Provision the VPC and the subnets

from {modules/network/main.tf}

```
resource "aws_vpc" "main"
resource "aws_subnet" "public"
resource "aws_subnet" "private"
```

Step 4 — Attach the internet gateway and configure routing

from {modules/network/main.tf}

```
resource "aws_internet_gateway" "igw"
resource "aws_route_table" "public"
resource "aws_route_table_association" "public"
```

Step 5 — Author the security groups

from {modules/network/main.tf}

```
resource "aws_security_group" "web" {
  name        = "${var.project_name}-web-sg"
  description = "HTTP for everyone and SSH only from the laboratory administrator CIDR"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "Public HTTP access to the web tier"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH only from the configured administrator address"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.admin_cidr]
  }

  egress {
    description = "Allow outbound package installation and responses"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.project_name}-web-sg"
  }
}

resource "aws_security_group" "db" {
  name        = "${var.project_name}-db-sg"
  description = "Database access only from members of the web security group"
  vpc_id      = aws_vpc.main.id

  ingress {
    description     = "MySQL/MariaDB from web tier only"
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.web.id]
  }

  egress {
    description = "Allow outbound response traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.project_name}-db-sg"
  }
}
```

Step 6 — Create the web tier using count

{modules/compute/main.tf}

```
resource "aws_instance" "web" {
  count = var.web_count

  ami                         = var.ami_id
  instance_type               = var.instance_type
  key_name                    = var.key_pair_name
  subnet_id                   = var.public_subnet_id
  vpc_security_group_ids      = [var.web_security_group_id]
  associate_public_ip_address = true
  user_data_replace_on_change = true

  metadata_options {
    http_endpoint = "enabled"
    http_tokens   = "required"
  }

  root_block_device {
    encrypted = true
  }

  user_data = <<-USERDATA
    #!/bin/bash
    set -euxo pipefail
    (dnf install -y httpd || yum install -y httpd)
    cat > /var/www/html/index.html <<'HTML'
    <!doctype html>
    <html><body><h1>System Provisioning Lab - Web Tier</h1><p>Provisioned by Terraform.</p></body></html>
    HTML
    systemctl enable --now httpd
  USERDATA

  tags = {
    Name = "${var.project_name}-web-${count.index + 1}"
    Tier = "web"
  }
}

# This instance has no public IP and is reachable on MySQL port 3306 only from
# the web security group. Use a database-ready AMI or a private package source
# if the lab also requires a running MySQL/MariaDB service.
resource "aws_instance" "db" {
  ami                         = var.ami_id
  instance_type               = var.instance_type
  key_name                    = var.key_pair_name
  subnet_id                   = var.private_subnet_id
  vpc_security_group_ids      = [var.db_security_group_id]
  associate_public_ip_address = false

  metadata_options {
    http_endpoint = "enabled"
    http_tokens   = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = "${var.project_name}-db-1"
    Tier = "database"
  }
}
```

Step 7 — Create the dependent database instance

From {modules/compute/main.tf}

```
resource "aws_instance" "db" {
  ami                         = var.ami_id
  instance_type               = var.instance_type
  key_name                    = var.key_pair_name
  subnet_id                   = var.private_subnet_id
  vpc_security_group_ids      = [var.db_security_group_id]
  associate_public_ip_address = false

  metadata_options {
    http_endpoint = "enabled"
    http_tokens   = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = "${var.project_name}-db-1"
    Tier = "database"
  }
}
```

Step 8 — Wire the modules together in the root module

From{main.tf}

main.tf

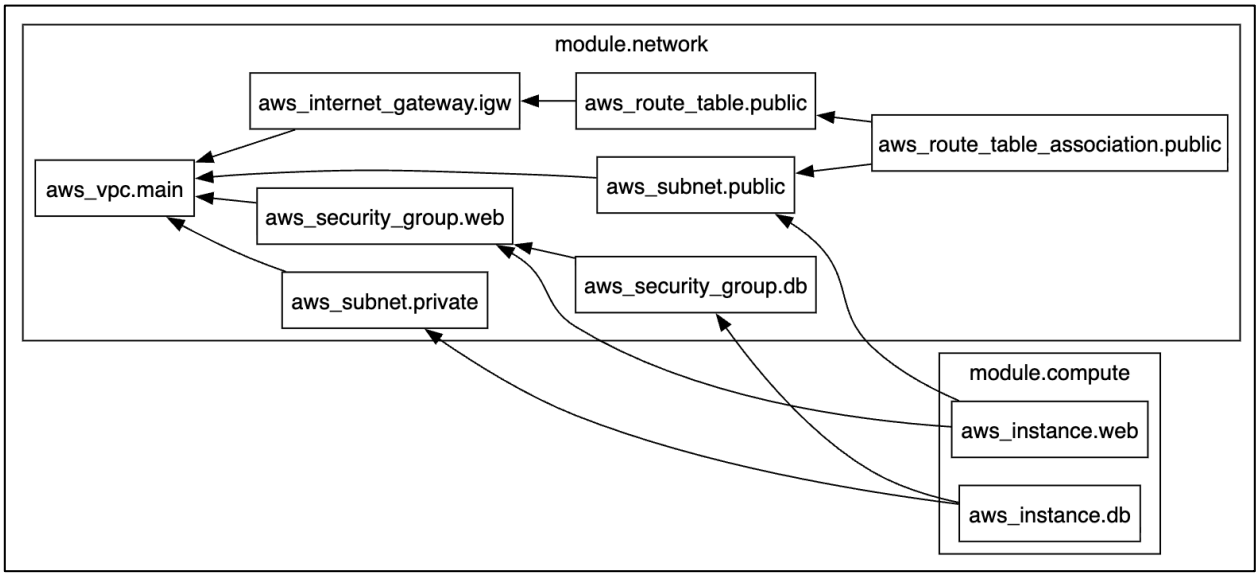
```
module "network" {
  source = "./modules/network"

  project_name        = var.project_name
  vpc_cidr            = var.vpc_cidr
  public_subnet_cidr  = var.public_subnet_cidr
  private_subnet_cidr = var.private_subnet_cidr
  availability_zone   = var.availability_zone
  admin_cidr          = var.admin_cidr
}

module "compute" {
  source = "./modules/compute"

  project_name          = var.project_name
  ami_id                = var.ami_id
  instance_type         = var.instance_type
  key_pair_name         = var.key_pair_name
  web_count             = var.web_count
  public_subnet_id      = module.network.public_subnet_id
  private_subnet_id     = module.network.private_subnet_id
  web_security_group_id = module.network.web_security_group_id
  db_security_group_id  = module.network.db_security_group_id
}
```

Step 9 — Inspect the dependency graph



Step 10 — Plan and apply the complete stack

```
      + cidr_block                          = "10.0.0.0/16"
      + default_network_acl_id              = (known after apply)
      + default_route_table_id              = (known after apply)
      + default_security_group_id           = (known after apply)
      + dhcp_options_id                     = (known after apply)
      + enable_dns_hostnames                = true
      + enable_dns_support                  = true
      + enable_network_address_usage_metrics = (known after apply)
      + id                                  = (known after apply)
      + instance_tenancy                    = "default"
      + ipv6_association_id                 = (known after apply)
      + ipv6_cidr_block                     = (known after apply)
      + ipv6_cidr_block_network_border_group = (known after apply)
      + main_route_table_id                 = (known after apply)
      + owner_id                            = (known after apply)
      + tags                                = {
          + "Name" = "spm-exp2-vpc"
        }
      + tags_all                            = {
          + "Experiment" = "Experiment-2"
          + "ManagedBy"  = "Terraform"
          + "Name"       = "spm-exp2-vpc"
          + "Project"    = "spm-exp2"
        }
    }

Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + db_private_ip  = (known after apply)
  + vpc_id         = (known after apply)
  + web_public_ips = [
      + (known after apply),
      + (known after apply),
    ]
```

```
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 % terraform apply tfplan
module.compute.aws_instance.db: Creating...
module.compute.aws_instance.web[1]: Creating...
module.compute.aws_instance.web[0]: Creating...
module.compute.aws_instance.db: Still creating... [00m10s elapsed]
module.compute.aws_instance.web[1]: Still creating... [00m10s elapsed]
module.compute.aws_instance.web[0]: Still creating... [00m10s elapsed]
module.compute.aws_instance.web[0]: Creation complete after 13s [id=i-00298c32519cdf041]
module.compute.aws_instance.web[1]: Creation complete after 14s [id=i-09407a98b64964b96]
module.compute.aws_instance.db: Creation complete after 14s [id=i-0e21f1013abde3a15]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

db_private_ip = "10.0.2.110"
vpc_id = "vpc-0f0915b2d66a83217"
web_public_ips = [
  "43.205.110.235",
  "52.66.203.208",
]
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 %
```

Step 11 — Execute the verification tests

```
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 % terraform output
db_private_ip = "10.0.2.110"
vpc_id = "vpc-0f0915b2d66a83217"
web_public_ips = [
  "43.205.110.235",
  "52.66.203.208",
]
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 %
```

both web servers are reachable-

```
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 % bash scripts/verify.sh
PASS: 43.205.110.235 returned HTTP 200
PASS: 52.66.203.208 returned HTTP 200
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 %
```

Database isolation-

```
PASS: 52.66.203.208 returned HTTP 200
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 % nc -vz -w 10 10.0.2.110 3306
nc: connectx to 10.0.2.110 port 3306 (tcp) failed: Operation timed out
```

Step 12 — Demonstrate reusability, then destroy

```
      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device {
          + delete_on_termination = true
          + device_name           = (known after apply)
          + encrypted             = true
          + iops                  = (known after apply)
          + kms_key_id            = (known after apply)
          + tags_all              = (known after apply)
          + throughput            = (known after apply)
          + volume_id             = (known after apply)
          + volume_size           = (known after apply)
          + volume_type           = (known after apply)
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  ~ web_public_ips = [
        # (1 unchanged element hidden)
        "52.66.203.208",
      + (known after apply),
    ]

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

module.network.aws_route_table_association.public: Destroying... [id=rtbassoc-099fa704efde2ffd0]
module.compute.aws_instance.web[1]: Destroying... [id=i-09407a98b64964b96]
module.compute.aws_instance.db: Destroying... [id=i-0e21f1013abde3a15]
module.compute.aws_instance.web[0]: Destroying... [id=i-00298c32519cdf041]
module.network.aws_route_table_association.public: Destruction complete after 1s
module.network.aws_route_table.public: Destroying... [id=rtb-01219f9337bbbe097]
module.network.aws_route_table.public: Destruction complete after 2s
module.network.aws_internet_gateway.igw: Destroying... [id=igw-0415388799fa464f2]
module.compute.aws_instance.db: Still destroying... [id=i-0e21f1013abde3a15, 00m10s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 00m10s elapsed]
module.compute.aws_instance.web[0]: Still destroying... [id=i-00298c32519cdf041, 00m10s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 00m10s elapsed]
module.compute.aws_instance.db: Still destroying... [id=i-0e21f1013abde3a15, 00m20s elapsed]
module.compute.aws_instance.web[0]: Still destroying... [id=i-00298c32519cdf041, 00m20s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 00m20s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 00m20s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 00m30s elapsed]
module.compute.aws_instance.web[0]: Still destroying... [id=i-00298c32519cdf041, 00m30s elapsed]
module.compute.aws_instance.db: Still destroying... [id=i-0e21f1013abde3a15, 00m30s elapsed]
module.compute.aws_instance.db: Destruction complete after 31s
module.network.aws_security_group.db: Destroying... [id=sg-0c3254a7104be7076]
module.network.aws_subnet.private: Destroying... [id=subnet-0ef4fcd73ff54b0ad]
module.compute.aws_instance.web[0]: Destruction complete after 31s
module.network.aws_subnet.private: Destruction complete after 1s
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 00m30s elapsed]
module.network.aws_security_group.db: Destruction complete after 2s
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 00m40s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 00m40s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 00m50s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 00m50s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m00s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m00s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m10s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m10s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m20s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m20s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m30s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m30s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m40s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m40s elapsed]
module.compute.aws_instance.web[1]: Still destroying... [id=i-09407a98b64964b96, 01m50s elapsed]
module.network.aws_internet_gateway.igw: Still destroying... [id=igw-0415388799fa464f2, 01m50s elapsed]
module.network.aws_internet_gateway.igw: Destruction complete after 1m51s
module.compute.aws_instance.web[1]: Destruction complete after 1m56s
module.network.aws_subnet.public: Destroying... [id=subnet-022c08e7de6de0f28]
module.network.aws_security_group.web: Destroying... [id=sg-02efdfc77c007688f]
module.network.aws_subnet.public: Destruction complete after 1s
module.network.aws_security_group.web: Destruction complete after 1s
module.network.aws_vpc.main: Destroying... [id=vpc-0f0915b2d66a83217]
module.network.aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 11 destroyed.
```

Program and Configuration Listings

Listing 2.1 — modules/network/main.tf (network objects)

Listing 2.2 — modules/network/main.tf (security groups)

Listing 2.3 — modules/compute/main.tf

Listing 2.4 — root main.tf and module wiring

Expected Output

```
Terraform will perform the following actions:

  # module.compute.aws_instance.db will be created
  + resource "aws_instance" "db" {
      + ami                                  = "ami-05a1f40ec1f9ea141"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = false
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "spm-exp2-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "spm-exp2-db-1"
          + "Tier" = "database"
        }
      + tags_all                             = {
          + "Experiment" = "Experiment-2"
          + "ManagedBy"  = "Terraform"
          + "Name"       = "spm-exp2-db-1"
          + "Project"    = "spm-exp2"
          + "Tier"       = "database"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options {
          + http_endpoint               = "enabled"
          + http_protocol_ipv6          = "disabled"
          + http_put_response_hop_limit = (known after apply)
          + http_tokens                 = "required"
          + instance_metadata_tags      = (known after apply)
        }
```

Output 2.1 — apply log showing the creation order

```
Outputs:

db_private_ip = "10.0.2.110"
vpc_id = "vpc-0f0915b2d66a83217"
web_public_ips = [
  "43.205.110.235",
  "52.66.203.208",
]
```

Output 2.2 — scaling test after changing web_count to 3

```
(base) shreyashshivhare@Shreyashs-MacBook-Air-3 exp-2 % terraform plan -var='web_count=3'
var.admin_cidr
  Your current public IPv4 address in CIDR notation. Only this range may SSH to the web tier.

  Enter a value: 103.212.138.244/32

var.ami_id
  AMI ID for an Amazon Linux web/database instance in aws_region.

  Enter a value: ami-05a1f40ec1f9ea141

var.key_pair_name
  Existing EC2 key-pair name in aws_region. The private .pem file is never stored in Terraform.

  Enter a value: spm-exp2-key

module.network.aws_vpc.main: Refreshing state... [id=vpc-0f0915b2d66a83217]
module.network.aws_internet_gateway.igw: Refreshing state... [id=igw-0415388799fa464f2]
module.network.aws_subnet.private: Refreshing state... [id=subnet-0ef4fcd73ff54b0ad]
module.network.aws_subnet.public: Refreshing state... [id=subnet-022c08e7de6de0f28]
module.network.aws_security_group.web: Refreshing state... [id=sg-02efdfc77c007688f]
module.network.aws_route_table.public: Refreshing state... [id=rtb-01219f9337bbbe097]
module.network.aws_route_table_association.public: Refreshing state... [id=rtbassoc-099fa704efde2ffd0]
module.network.aws_security_group.db: Refreshing state... [id=sg-0c3254a7104be7076]
module.compute.aws_instance.web[1]: Refreshing state... [id=i-09407a98b64964b96]
module.compute.aws_instance.web[0]: Refreshing state... [id=i-00298c32519cdf041]
module.compute.aws_instance.db: Refreshing state... [id=i-0e21f1013abde3a15]

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  ~ web_public_ips = [
        # (1 unchanged element hidden)
        "52.66.203.208",
      + (known after apply),
    ]
```

My Experiment Output

Observation Table

Complete this table with the values observed during your own session.

Test	Method	Expected	Observed
Creation order		Network before compute	VPC was created first, followed by subnets, internet gateway, route table, and security groups; web and DB instances were created afterward.
Web reachability		HTTP 200 from both	43.205.110.235 returned HTTP 200 and 52.66.203.208 returned HTTP 200.
SSH restriction		Connection times out	Timed out after 1_0_s
Database isolation		Refused or timed out	ssh: connect to host 43.205.110.235 port 22: Operation timed out
Tier-to-tier access		Connection succeeds	The current DB VM has no MariaDB/MySQL service running, so the port test cannot prove an application connection.
Reusability		1 to add, 0 to change	Plan: 1 to add, 0 to change, 0 to destroy.
Teardown order		Instances before subnets	Web and DB instances were removed before security groups, subnets, route table, gateway, and VPC.

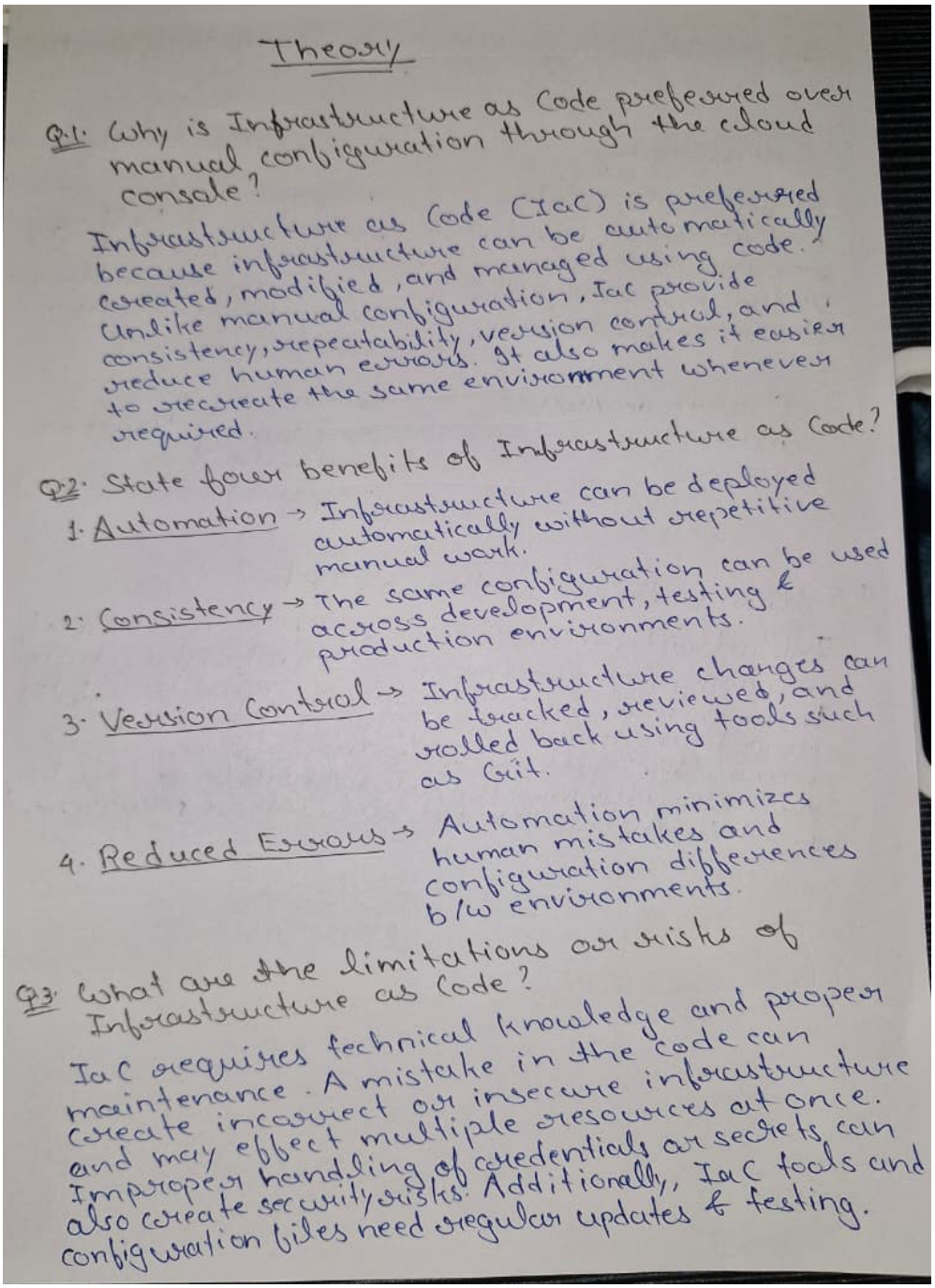
Result

A two-tier network topology consisting of a VPC, a public and a private subnet, an internet gateway, routing and two security groups, together with three interdependent virtual machines, …

Precautions

- Set the admin CIDR to the laboratory's actual public address. Leaving SSH open to 0.0.0.0/0 will cause the configuration to be rejected during evaluation and exposes the instance to automated attacks within minutes.
- Place the database in the private subnet. An instance in a public subnet with a restrictive security group is still one misconfiguration away from exposure.
- Do not embed the key pair file or its contents in the configuration; reference the key by name only.
- Verify that every subnet CIDR lies within the VPC CIDR and that the two subnets do not overlap.

Theory-



Theory

Q.1. Why is Infrastructure as Code preferred over manual configuration through the cloud console?

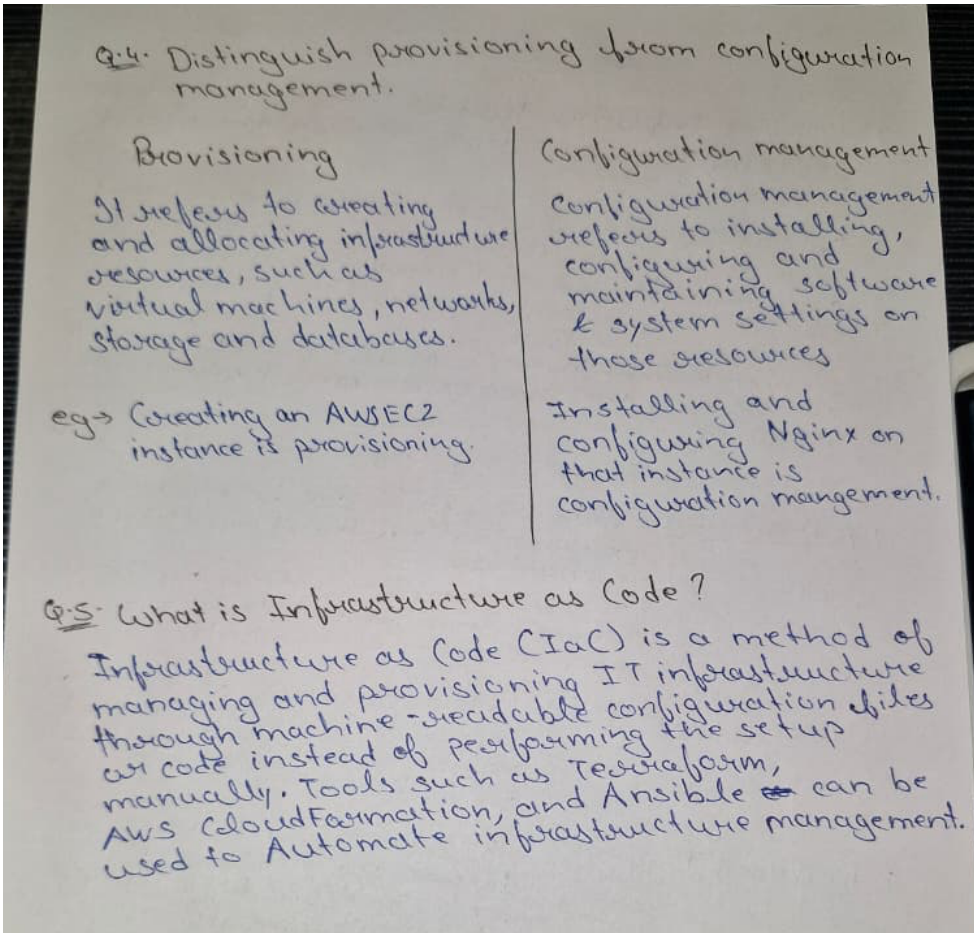
Infrastructure as Code (IaC) is preferred because infrastructure can be automatically created, modified, and managed using code. Unlike manual configuration, IaC provide consistency, repeatability, version control, and reduce human errors. It also makes it easier to recreate the same environment whenever required.

Q.2. State four benefits of Infrastructure as Code?

1. Automation → Infrastructure can be deployed automatically without repetitive manual work.
2. Consistency → The same configuration can be used across development, testing & production environments.
3. Version Control → Infrastructure changes can be tracked, reviewed, and rolled back using tools such as Git.
4. Reduced Errors → Automation minimizes human mistakes and configuration differences b/w environments.

Q.3. What are the limitations or risks of Infrastructure as Code?

IaC requires technical knowledge and proper maintenance. A mistake in the code can create incorrect or insecure infrastructure and may effect multiple resources at once. Improper handling of credentials or secrets can also create security risks. Additionally, IaC tools and configuration files need regular updates & testing.



Q.4. Distinguish provisioning from configuration management.

Provisioning	Configuration management
It refers to creating and allocating infrastructure resources, such as virtual machines, networks, storage and databases.	Configuration management refers to installing, configuring and maintaining software & system settings on those resources
eg→ Creating an AWS EC2 instance is provisioning.	Installing and configuring Nginx on that instance is configuration mangement.

Q.5. What is Infrastructure as Code?

Infrastructure as Code (IaC) is a method of managing and provisioning IT infrastructure through machine-readable configuration files or code instead of performing the setup manually. Tools such as Terraform, AWS CloudFormation, and Ansible can be used to Automate infrastructure management.